

Sklep internetowy z kawą — dokumentacja techniczna

Autorzy: Hubert Myszka, Michał Nowak

Projekt jest sklepem internetowym z kawą, akcesoriami i ekspresami. Całość zbudowana jest wokół relacyjnej bazy danych PostgreSQL, w której logika biznesowa (walidacje, transakcje, raporty) jest zrealizowana po stronie bazy — w postaci procedur składowanych, funkcji, widoków i wyzwalaczy. Warstwa aplikacyjna (Spring Boot) pełni rolę cieniowego pośrednika, który wywołuje te obiekty bazodanowe i wystawia je jako REST API, a frontend (React) konsumuje to API.

1. Wykorzystywane technologie

Warstwa	Technologia	Wersja / uwagi
Baza danych	PostgreSQL	16 (obraz <code>postgres:16</code> w Dockerze)
Język logiki bazodanowej	PL/pgSQL	procedury, funkcje, wyzwalacze
Backend	Java	25
Framework backendu	Spring Boot	4.0.x (Spring Web MVC, Spring Data JPA)
ORM	Hibernate / JPA	dostęp do bazy, mapowanie encji
Bezpieczeństwo haseł	Spring Security Crypto	hashowanie haseł (BCrypt)
Redukcja boilerplate	Lombok	getter/setter/buildery
Build backendu	Maven	(wrapper <code>mvnw</code>)
Frontend	React	19 + TypeScript
Build/dev frontendu	Vite	8
Routing frontendu	React Router	7
Konteneryzacja bazy	Docker / Docker Compose	
Testy API	Postman	kolekcja <code>Postman_Collection.json</code>

Kluczowa decyzja architektoniczna — logika w bazie

Świadomie przyjęliśmy podejście „**database-first**”: reguły biznesowe i integralność danych są egzekwowane w bazie, a nie w aplikacji. Backend praktycznie nie zawiera logiki — w większości przypadków metoda repozytorium to po prostu wywołanie procedury lub funkcji przez natywne zapytanie SQL.

Zalety, które chcieliśmy uzyskać:

- Spójność niezależnie od klienta** — te same reguły obowiązują, czy operacja przyjdzie z REST API, z konsoli `psql`, czy z testu. Nie da się ich obejść przez inną ścieżkę dostępu.
- Atomowość** — operacje wieloetapowe (np. opłacenie zamówienia) wykonują się w jednej transakcji bazodanowej.
- Wydajność** — walidacje i agregacje dzieją się tam, gdzie są dane, bez przysyłania ich do aplikacji.

Hibernate jest tu skonfigurowany w trybie `spring.jpa.hibernate.ddl-auto=validate` — ORM **nie** generuje schematu. Schemat tworzą skrypty SQL, a Hibernate jedynie weryfikuje, że encje Javy zgadzają się z istniejącymi tabelami.

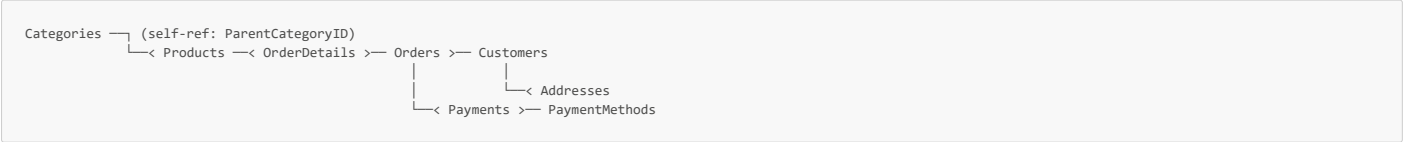
2. Model / schemat bazy danych

Schemat opisuje typową domenę e-commerce: katalog produktów z hierarchią kategorii, klientów z adresami, zamówienia ze szczegółami pozycji oraz płatności. Pełny diagram ER znajduje się w [db/erd/Entity-Relationship Diagram.pdf](#), a jego źródło ([dbdiagram.io](#)) w [db/erd/](#).

Tabele

- Categories** — kategorie produktów. Kolumna `ParentCategoryID` jest kluczem obcym wskazującym na tę samą tabelę (`Categories_fk1`), co daje **samoodwołującą się hierarchię** (drzewo kategorii). Np. „Kawa” → „Kawa ziarnista”.
- Products** — produkty. `CategoryID` → `Categories`. Cena `decimal(10,2)`, stan magazynowy `Stock`, flaga `IsActive` (miękkie usuwanie) oraz kolumna `Attributes` typu `jsonb` na atrybuty zależne od kategorii (np. waga i stopień palenia dla kawy, ciśnienie i średnica kolby dla ekspresu).
- Customers** — klienci. `Email` z ograniczeniem `UNIQUE`, opcjonalny `PasswordHash` (konta założone z danych seedowych mogą nie mieć hasła).
- Addresses** — adresy klienta. `CustomerID` → `Customers`. Flagi `IsActive` (miękkie usuwanie) i `IsDefault` (jeden adres domyślny na klienta).
- Orders** — zamówienia. `CustomerID` → `Customers`, `AddressID` → `Addresses`. `Status` przyjmuje wartości `pending` / `packed` / `delivered` / `cancelled`.
- OrderDetails** — pozycje zamówienia, tabela łącząca M:N między `Orders` a `Products`. **Klucz główny złożony** (`OrderID`, `ProductID`). Przechowuje `UnitPrice` z chwili zakupu (cena historyczna, niezależna od późniejszych zmian ceny produktu) oraz `Quantity`.
- PaymentMethods** — metody płatności (BLIK, Visa, PayPal...), z flagą `IsActive`.
- Payments** — płatności. `OrderID` → `Orders`, `PaymentMethodID` → `PaymentMethods`. `Status`, `CreatedAt`, `PaidAt`.

Relacje w skrócie



Zastosowane techniki modelowania

- Hierarchia rekurencyjna** — drzewo kategorii zbudowane jako tabela odwołująca się do siebie; przechodzone rekurencyjnym CTE (sekcja 3).
- jsonb zamiast EAV** — atrybuty produktów są nieregularne (każda kategoria ma inny zestaw), więc zamiast modelu Entity-Attribute-Value używamy kolumny `jsonb`. Pozwala to filtrować po atrybutach operatorem zawierania `@>` i indeksować je w razie potrzeby (GIN).
- Miękkie usuwanie** (`IsActive`) — produktów, adresów i metod płatności nie kasujemy fizycznie (psułoby to historyczne zamówienia), tylko dezaktywujemy. Widoki `vw_active_*` udostępniają tylko aktywne rekordy.
- Ceny historyczne** — `OrderDetails.UnitPrice` zamraża cenę z momentu złożenia zamówienia.
- GENERATED ALWAYS AS IDENTITY** — nowoczesny, zgodny ze standardem SQL sposób generowania kluczy głównych (zamiast `SERIAL`).

3. Realizacja operacji w bazie danych

Plik schematu i wszystkie obiekty bazodanowe są ładowane automatycznie przy starcie kontenera z katalogu `db/init/` (montowanego do `docker-entrypoint-initdb.d`), w kolejności:

Plik	Zawartość
<code>01_schema.sql</code>	tabele, klucze główne i obce
<code>02_views.sql</code>	widoki
<code>03_procedures.sql</code>	procedury składowane

Plik	Zawartość
04_functions.sql	funkcje
05_triggers.sql	wyzwalacze
06_data.sql	dane przykładowe (seed)

Poniżej przegląd wykorzystanych mechanizmów PostgreSQL wraz z komentarzem.

3.0 Trzy wymagane kategorie operacji

Projekt realizuje wszystkie trzy typy operacji wymagane w specyfikacji:

Kategoria	Przykładowe operacje w projekcie	Realizacja
Proste operacje CRUD	dodanie/zmiana/dezaktywacja produktu, zmiana ceny i stanu, dodanie/usunięcie adresu, rejestracja i edycja klienta, odczyt produktów, kategorii, zamówień, płatności	procedury <code>p_add_*</code> , <code>p_set_*</code> , <code>p_deactivate_*</code> , <code>p_change_*</code> ; odczyty przez widoki i funkcje <code>f_get_*</code>
Złożone operacje transakcyjne (kontrola zasobów i współbieżności)	opłacenie zamówienia (<code>p_pay_order</code>) — zdjęcie stanu magazynowego „limitowanego” zasobu, zmiana statusu i utworzenie płatności atomowo; złożenie zamówienia (<code>p_create_order</code>); anulowanie ze zwrotem stanu (trigger)	procedury PL/pgSQL + blokady <code>FOR UPDATE</code> + wyzwalacze — szczegóły w sekcji 3.8
Operacje raportujące (złożone zapytania, łączenie tabel, agregacja)	bestsellery (<code>f_report_best_sellers</code>), przychód wg kategorii (<code>f_report_revenue_by_category</code>), suma wydatków klienta (<code>f_get_customer_expenses</code>)	funkcje z <code>JOIN</code> , <code>GROUP BY</code> , <code>SUM</code> , rekurencyjnym CTE i <code>LEFT JOIN</code>

3.1 Widoki

Trzy widoki (`vw_active_products`, `vw_active_paymentmethods`, `vw_active_addresses`) hermetyzują filtr `IsActive = true`. Dzięki temu reszta kodu (procedury, repozytoria) nie powtarza tego warunku i nie ryzykuje przypadkowego pokazania zdezaktywowanego rekordu.

3.2 Procedury składowane (CALL)

Procedury realizują operacje **modyfikujące stan** i obejmują walidację + zmianę danych w jednej transakcji. Wydzieliliśmy zbiór małych procedur-strażników (`p_check_*`), które rzucają wyjątek, gdy warunek nie jest spełniony, i są wywoływane z procedur głównych — to ponowne użycie kodu walidacyjnego zamiast kopiowania go w każdym miejscu:

```
CREATE OR REPLACE PROCEDURE p_check_address_belongs_to_customer(p_addressid int, p_customerid int) AS $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM addresses
                   WHERE addressid = p_addressid AND customerid = p_customerid) THEN
        RAISE EXCEPTION 'Address with ID % does not belong to Customer with ID %', p_addressid, p_customerid;
    END IF;
END;
$$ LANGUAGE plpgsql;
```

Najważniejsze procedury operacyjne:

- `p_create_order(customerid, addressid, items jsonb)` — tworzy zamówienie. Przyjmuje pozycje jako tablicę `JSON`, którą rozwija `jsonb_array_elements`, i dla każdej pozycji sprawdza istnienie i aktywność produktu, po czym wstawia wiersz do `orderdetails` z ceną pobraną z `products`. Przekazywanie listy pozycji jako jednego argumentu `jsonb` pozwala utworzyć całe zamówienie jednym `CALL`.
- `p_pay_order(customerid, orderid, paymentmethodid)` — opłaca zamówienie. To najbardziej złożona operacja: waliduje własność i status zamówienia, **blokuje wiersze pozycji** (`FOR UPDATE`), zdejmując stan magazynowy każdego produktu (przez `p_update_stock`), zmienia status na `packed` i wstawia rekord płatności z obliczoną kwotą. Wszystko atomowo — albo całość się powiedzie, albo nic.
- `p_update_stock(productid, quantity)` — zmienia stan magazynowy z blokadą `SELECT ... FOR UPDATE` i kontrolą, że stan nie zejdzie poniżej zera (`RAISE EXCEPTION 'Stock cannot be negative'`).
- `p_register_customer(...)` — rejestracja z `INSERT ... ON CONFLICT (email) DO UPDATE (upsert)`, pozwalająca „dokończyć” konto seedowe bez hasła, ale blokująca nadpisanie konta z hasłem.
- `p_add_address, p_set_default_address, p_deactivate_address` — zarządzanie adresami z utrzymaniem niezmiennika „dokładnie jeden adres domyślny”: dezaktywacja adresu domyślnego automatycznie przenosi flagę na inny aktywny adres.
- `p_change_order_status(orderid, newstatus)` — zmiana statusu z walidacją dozwolonej wartości i automatycznym ustawieniem `ShipDate` przy przejściu do `delivered`.

3.3 Funkcje

Funkcje realizują operacje **odczytu / raportowania** i zwracają zbiory wierszy (`RETURNS TABLE` / `RETURNS SETOF`).

- `f_get_category_subtree(category_id)` — zwraca identyfikatory danej kategorii i wszystkich jej podkategorii za pomocą **rekurencyjnego CTE** (`WITH RECURSIVE`). To serce obsługi hierarchii — używane przez filtrowanie i raporty, by „kategoria nadrzędna” obejmowała wszystkie potomne.

```
WITH RECURSIVE category_tree AS (
    SELECT c.categoryid FROM categories c WHERE c.categoryid = f_category_id
    UNION ALL
    SELECT c.categoryid FROM categories c
    INNER JOIN category_tree ct ON c.parentcategoryid = ct.categoryid
)
SELECT ct.categoryid FROM category_tree ct;
```

- `f_filter_products(min_price, max_price, category_id, attributes_filter jsonb)` — wyszukiwarka produktów. Każdy argument jest opcjonalny (wzorec (`param IS NULL OR warunek`)), filtruje po przedziale cenowym, po poddrzewie kategorii oraz **po atrybutach jsonb operatorem zawierania @>**.
- `f_report_best_sellers(limit, category_id)` — raporty bestsellerów: agreguje sprzedane sztuki i przychód (`SUM, GROUP BY`), liczy tylko zamówienia opłacone/dostarczone, opcjonalnie zawęza do poddrzewa kategorii.
- `f_report_revenue_by_category()` — przychód w rozbiciu na kategorie, z `LEFT JOIN`, by pokazać też kategorie bez sprzedaży (przychód 0).
- Funkcje pomocnicze klienta: `f_get_customer_expenses`, `f_get_customer_orders`, `f_get_customer_addresses`, `f_get_customer_payments`, `f_get_order_total`, `f_get_order_products`.

3.4 Wyzwalacze (trigger)

Wyzwalacze pilnują niezmienników, których nie da się wyrazić prostym ograniczeniem `CHECK` — wymagają zajrzenia do innych wierszy/tabel:

- `t_validate_leaf_category` (BEFORE INSERT/UPDATE na `products`) — produkt można przypisać tylko do kategorii-liścia (nieposiadającej podkategorii).
- `t_validate_product_active` (BEFORE INSERT na `orderdetails`) — nie można dodać do zamówienia nieaktywnego produktu.
- `t_validate_payment_method_active` (BEFORE INSERT na `payments`) — płatność tylko aktywną metodą.
- `trg_validate_address_active` (BEFORE INSERT/UPDATE na `orders`) — zamówienie tylko na aktywny adres.
- `trg_validate_order_status_transition` (BEFORE UPDATE na `orders`) — **maszyna stanów** zamówienia: dopuszcza tylko sensowne przejścia statusu (np. blokuje `delivered` → `pending` czy zmianę statusu zamówienia anulowanego).
- `trg_restore_stock_on_cancel` (AFTER UPDATE na `orders`) — przy anulowaniu zamówienia spakowanego (`packed` → `cancelled`) **automatycznie zwraca zdjęty stan magazynowy**.

3.5 Demonstracja możliwości i dyskusja zastosowanych technik

Projekt celowo pokazuje szeroki wachlarz mechanizmów PostgreSQL:

- Typ jsonb i operator @>** — elastyczne, indeksowalne atrybuty produktów bez rozbudowywania schematu; zamówienia przyjmowane jako `JSON` i parsowane w bazie.
- Rekurencyjne CTE** — przechodzenie drzewa kategorii niezależnie od głębokości.
- Współbieżność i blokady (FOR UPDATE)** — w `p_update_stock` i `p_pay_order` blokujemy wiersze produktów, by dwie równoległe płatności nie sprzedały tej samej, ostatniej sztuki. Poprawność tego rozwiązania jest pokryta testem współbieżnościowym (sekcja 5).
- Procedury vs funkcje** — świadomy podział: procedury (`CALL`), mogą zarządzać transakcją, modyfikują stan) do zapisu, funkcje (`SELECT`, zwracają zbiory) do odczytu.
- Strażnicy p_check_*** — walidacja jako zestaw małych, wielokrotnie używanych procedur; pojedyncze źródło prawdy dla każdej reguły.

- **Sygnalizowanie błędów przez `RAISE EXCEPTION`** — naruszenie reguły zwraca czytelny komunikat, który backend mapuje na odpowiedź HTTP.
- **Miękkie usuwanie + widoki** — historia pozostaje nienaruszona, a kod aplikacyjny widzi tylko aktywne rekordy.

3.6 Sposób wywołania z aplikacji

Backend nie duplikuje tej logiki — repozytoria Spring Data wywołują obiekty bazodanowe natywnym SQL. Przykłady:

```
// procedura modyfikująca - OrderRepository
@Transactional
@Query(value = "CALL p_create_order(:customerId, :addressId, CAST(:items AS jsonb))", nativeQuery = true)
void createOrder(@Param("customerId") Integer customerId,
                 @Param("addressId") Integer addressId,
                 @Param("items") String items);

// funkcja zwracająca tabelę - ProductRepository
@Query(value = "SELECT * FROM f_report_best_sellers(:limit, :categoryId)", nativeQuery = true)
List<Object[]> findBestSellers(@Param("limit") Integer limit, @Param("categoryId") Integer categoryId);

// odczyt przez widok
@Query(value = "SELECT * FROM vw_active_products", nativeQuery = true)
List<Product> findAllActive();
```

Lista pozycji zamówienia jest w `OrderService` serializowana do JSON-a (Jackson) i przekazywana do `p_create_order` jako `jsonb`. Hasła są hashowane w warstwie aplikacji (`PasswordEncoder` / BCrypt) przed wywołaniem `p_register_customer`.

3.7 Przetwarzanie transakcyjne, współbieżność i wydajność

To jest centralny punkt projektu — sterowanie dostępem do „limitowanego” zasobu, jakim jest stan magazynowy produktu.

Transakcyjność. Operacje wieloetapowe są zamknięte w pojedynczej transakcji. `p_pay_order` w jednym wywołaniu: waliduje zamówienie, dla każdej pozycji zdejmuje stan, zmienia status zamówienia i wstawia płatność. Jeśli którykolwiek krok rzuci wyjątek (np. brak stanu), **cała transakcja jest wycofywana** (`ROLLBACK`) — nie powstaje płatność za zamówienie, którego nie da się zrealizować, ani nie zostaje zdjęty częściowy stan. Po stronie aplikacji granicę transakcji wyznacza `@Transactional` na metodzie repozytorium.

Kontrola równoczesnego dostępu. Scenariusz wyjścia: dwóch klientów jednocześnie płaci za ostatnią sztukę produktu. Bez ochrony oba odczyty zobaczyłyby `stock = 1` i oba zapisy zeszyłyby do `0/-1` (nadsprzedaż). Rozwiązanie w `p_update_stock`:

```
SELECT stock INTO v_stock
FROM products
WHERE productid = f_product_id
FOR UPDATE;           -- blokada wiersza do końca transakcji

IF v_stock + quantity < 0 THEN
  RAISE EXCEPTION 'Stock cannot be negative';
END IF;
```

Klauzula `FOR UPDATE` zakłada blokadę na poziomie pojedynczego wiersza produktu. Druga transakcja czeka, aż pierwsza się zakończy, a następnie widzi już zaktualizowany stan i jej kontrola `v_stock + quantity < 0` poprawnie ją odrzuca. Jest to blokada wierszowa (a nie tabeli), więc operacje na różnych produktach nie blokują się nawzajem. Poprawność tego mechanizmu potwierdza test `StockConcurrencyTest` (sekcja 5): z dwóch równoległych płatności dokładnie jedna kończy się sukcesem, druga otrzymuje `Stock cannot be negative`, a stan końcowy wynosi 0 — brak nadsprzedaży.

Wydajność. Zastosowane techniki ograniczają koszt operacji:

- **Logika wykonywana przy danych** — walidacje, agregacje i pętle dzieją się w bazie; pojedyncze zamówienie powstaje jednym `CALL` zamiast wieloma round-tripami aplikacja↔baza.
- **Blokady wierszowe zamiast tabelowych** (`FOR UPDATE`) — minimalna sekcja krytyczna, wysoka współbieżność dla różnych produktów.
- **Operacje zbiorowe** — raporty liczą się jednym zapytaniem z `GROUP BY/JOIN`, bez pobierania surowych wierszy do aplikacji.
- **Indeksy** — klucze główne i `UNIQUE (Email)` są automatycznie indeksowane; klucze obce wspierają złączenia. Kolumnę `Attributes (jsonb)` można w razie potrzeby objąć indeksem `GIN`, by przyspieszyć filtrowanie operatorem `@>`.

3.8 REST API (wystawienie operacji)

Operacje bazodanowe są udostępniane jako REST. Najważniejsze endpointy:

Metoda	Ścieżka	Operacja
POST	/api/auth/login, /logout, GET /me	logowanie / sesja
POST	/api/customers	rejestracja klienta
GET	/api/products	aktywne produkty
POST	/api/products/filter	filtrowanie (cena, kategoria, atrybuty <code>jsonb</code>)
PATCH	/api/products/{id}/deactivate, /updatePrice, /addStock	zarządzanie produktem
GET	/api/categories, /api/categories/{id}	kategorie
POST/GET	/api/addresses	adresy klienta
POST	/api/orders	złożenie zamówienia
GET	/api/orders	zamówienia klienta
POST	/api/orders/{id}/pay, /cancel, /deliver	cykl życia zamówienia
GET	/api/payments, /api/payments/total	płatności i suma wydatków
GET	/api/reports/best-sellers, /best-sellers/by-category	raporty

4. Struktura repozytorium

```
.
├── docker-compose.yml      # kontener PostgreSQL 16
├── db/
│   ├── init/              # skrypty ładowane przy starcie bazy (01-06)
│   ├── erd/               # diagram ER (PDF + źródło dbdiagram.io)
├── backend/                # Spring Boot (Java 25, Maven)
│   ├── src/main/java/.../backend
│   │   ├── entity/        # encje JPA (mapowanie tabel)
│   │   ├── repository/    # repozytoria - wywołania CALL / SELECT funkcji
│   │   ├── service/       # cienka warstwa serwisowa
│   │   ├── web/           # kontrolery REST
│   │   ├── dto/           # obiekty transferowe
│   │   └── config/        # konfiguracja (hasła, obsługa wyjątków)
```

```
|— frontend/                # React 19 + TypeScript + Vite
|— Postman_Collection.json  # przykładowe żądania API
```

5. Testy

W `backend/src/test` znajdują się testy integracyjne uruchamiane na realnej bazie:

- `BackendApplicationTest` — sprawdza połączenie z bazą i obecność wszystkich tabel.
- `DatabaseOperationsTest` — testuje logikę bazodanową: zdejmowanie stanu przy płatności, odrzucenie płatności przy niewystarczającym stanie, poprawność raportu bestsellerów (z pominięciem zamówień nieopłaconych), blokadę zamówienia na cudzy adres.
- `StockConcurrencyTest` — **test współbieżności**: dwie równoległe płatności o ostatnią sztukę produktu; weryfikuje, że dokładnie jedna się powiedzie, druga dostanie `Stock cannot be negative`, a stan końcowy wyniesie 0 (brak nadsprzedaży dzięki `FOR UPDATE`).

Uruchomienie: `cd backend && ./mvnw test` (wymaga działającej bazy).

6. Instrukcja uruchomienia

Wymagania

- Docker + Docker Compose
- JDK 25
- Node.js (dla frontendu)

1. Baza danych

W katalogu głównym projektu:

```
docker compose up -d
```

Uruchamia PostgreSQL 16 na porcie **5432** (baza `shopdb`, użytkownik `shopuser`, hasło `shoppassword`) i automatycznie wykonuje wszystkie skrypty z `db/init/` (schemat, widoki, procedury, funkcje, trigger, dane przykładowe).

Skrypty inicjalizacyjne wykonują się **tylko przy pierwszym** tworzeniu wolumenu. Aby przeładować bazę od zera:

```
docker compose down -v && docker compose up -d
```

2. Backend

```
cd backend
./mvnw spring-boot:run      # Windows: mvnw.cmd spring-boot:run
```

Backend startuje na `http://localhost:8080`. Dane połączenia są w `backend/src/main/resources/application.properties` (zgodne z `docker-compose.yml`). Hibernate działa w trybie `validate` — wymaga, by baza była już zainicjalizowana skryptami.

3. Frontend

```
cd frontend
npm install
npm run dev
```

Aplikacja będzie dostępna pod adresem podanym przez Vite (domyślnie `http://localhost:5173`).

4. Testowanie API

Można zaimportować `Postman_Collection.json` do Postmana aby zweryfikować poprawność metod i funkcjonalności, które nie mają frontendu, ponieważ jest to czysta logika biznesowa nie związana z samym sklepem internetowym. Ustawić zmienną `baseUrl` na `http://localhost:8080`.